

多 Agent 异构资源博弈管理系统实践报告

231240045 杨俊炜

2026/6/11

1 背景知识与问题提出

1.1 背景知识

在当前大语言模型（LLM）推理的底层物理运行中，不同的硬件单元承担着截然不同的职责：

- **CPU（中央处理器）**：负责逻辑控制、数据 I/O 与前置的 Tokenization（数据分词）。发热中等。
- **GPU（图形处理器）**：大模型推理的主力算力节点，主导极度消耗资源的 Prefill（预填充）阶段和大规模注意力矩阵乘法。功耗极高，极易触发温度墙。
- **NPU（神经网络处理器）**：专为 AI 设计的底层加速硬件。能在极低功耗和低发热下完成端侧的小模型推理或接管解码（Decoding）任务。

1.2 问题提出

针对在家中多台旧设备上部署计算任务的场景，传统调度仅仅将负载泛化为单一的“CPU 占用”。这种一刀切的模型无法反映 AI 时代异构硬件的真实物理属性。当一台边缘设备接取 LLM 推理任务时，若缺乏对不同硬件特性的认知，极易导致 GPU 过热宕机，或造成低功耗 NPU 资源的闲置浪费。

2 现状：场景、工具与参考集

- **目前场景**：模拟在家中闲置的多台异构旧设备（如旧笔记本、旧手机平板）组成的边缘计算集群。每台设备均具备不同程度的 CPU、GPU 和 NPU 算力。
- **使用工具**：Python，并引入分布式计算框架 **Ray** 实现多节点的异步状态同步与 RPC 调用。
- **参考集**：借鉴了 LangChain/AutoGPT 的 Agent 协作思想，并参考了《星际争霸》中的微观资源运营机制，引入了“电费 Token”作为衡量算力消耗的一般等价物。

3 我的工作：目标、实现与核心机制

3.1 1. 工作目标

设计并实现一个具备“软硬结合”思维的多 Agent 资源博弈调度系统。核心是让 Master Agent 能够识别并调控 Worker Agent 内部 CPU、GPU、NPU 的异构负载特征，通过 Token 经济学实现整体集群的功耗平衡与算力最大化。

3.2 2. 具体实现（核心博弈机制与数学模型）

系统采用 **Master-Worker**（主从架构）。节点状态被扩展为一个七维向量以支持异构计算：

$$S_i^{(t)} = \langle T_i^{(t)}, C_i^{(t)}, G_i^{(t)}, N_i^{(t)}, M_i^{(t)}, A_i^{(t)}, R_i^{(t)} \rangle \quad (1)$$

其中新增了 $G_i^{(t)}$ (GPU 占用率) 与 $N_i^{(t)}$ (NPU 占用率)。

Master 的调度由抢占阈值 $\tau = 10$ 决定：

1. **饥饿打工 (NPU 边缘推理)**: 当 $T_i < 10$ 时，设备被迫转入低功耗状态，关闭 GPU，利用 NPU 高效赚取 Token ($\Delta T_{\text{earn}} \in [4, 7]$)，同时物理降温。
2. **重载抢占 (异构任务分发)**: 当 $T_i \geq 10$ 时，接取任务：
 - **LLM Prefill (GPU)**: $G_i \in [85, 99]$ ，极度消耗 Token (-12)，温度剧增 ($+20 \sim 35^\circ\text{C}$)。
 - **Data Tokenization (CPU)**: $C_i \in [85, 98]$ ，中等消耗 Token (-8)，温度中等上升。
3. **阶梯惩罚函数 (Stick Policy)**: 针对异构硬件特性更新了惩罚逻辑。

$$P(S_i^{(t)}) = \begin{cases} 15, & \text{if } M_i^{(t)} > 85.0 \text{ (整体过热)} \\ 10, & \text{if } G_i^{(t)} > 95.0 \text{ (GPU 撞墙)} \\ 5, & \text{if } C_i^{(t)} > 95.0 \text{ (CPU 满载)} \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

4 实验：设计、验证与结果分析

4.1 1. 实验环境与配置

- **实验环境**: 本地单机环境，Python 3.10，Ray 框架。
- **实验配置**: 部署 1 个 Master Agent 接管 3 台异构 Worker 节点。博弈周期 (Rounds) 设为 10 轮。节点初始 Token 均为 0。

4.2 2. 预验证的目标

1. 验证“饥饿机制”是否能有效驱使节点通过 NPU 低功耗运行完成冷启动资本积累。2. 验证 GPU 和 CPU 重负载任务对 Token 消耗与温度攀升的差异性影响。3. 验证 Master 能否精确捕捉并执行阶梯状的热力惩罚。

4.3 3. 实验结果与分析

第一阶段 (R1-R2): NPU 冷启动。所有节点因 Token 不足触发饥饿机制，全面调用 NPU 进行边缘推理，GPU 占用率归零。系统整体温度处于 35°C 安全水位，Token 成功突破抢占阈值。

第二阶段 (R3-R7): 算力抢占与异构分离。节点开始分化。接取 LLM Prefill 的节点 (如 Node-1)，其 GPU 占用飙升至 95% 以上，温度突破 80°C ；而接取 Data Tokenization 的节点，CPU 飙升但温度上升曲线相对平缓。

第三阶段（R8-R10）：阶梯惩罚与热力控制。由于 GPU 的持续重载，部分节点触发 $M > 85.0$ 和 $G > 95.0$ 的双重红线，被 Master 强行扣除大量 Token 并归零。这些节点随后被迫长期进入 NPU 待机状态进行物理散热，直到温度回落才重新参与博弈。

5 系统状态博弈循环流图

以下为系统在单个调度周期（Round）内的状态循环逻辑流：

表 1: Master-Worker 回合博弈流水线

步骤	执行主体	核心行为
Step A	Master Agent	结算上一轮积分奖励；检测温度、CPU 并执行过热罚款。
Step B	Master → Worker	拉取最新状态，根据 $T < 10$ 判别式下发低功耗或重载任务。
Step C	Worker Agent (Ray)	异步执行。打工节点降温续航；核心抢占节点消耗 Token 冲高分。
Step D	全局结算面板	打印当前轮次的 Tokens、温度、负载及收益看板。

6 实践中遇到的困难与解决办法

6.1 挑战 1：单机环境下的分布式调度与状态隔离

实践困难：在实验初期的代码编写阶段，首要难点是如何在单台个人笔记本电脑上，模拟出多台物理隔离的异构老旧设备并发运行，并对它们的状态进行统一、非阻塞的监控与追踪。传统的 Python 多线程/多进程库编写繁琐，且难以实现跨节点的对象状态同步。

解决办法：放弃了底层并发库，选择引入成熟的 **Ray** 框架实现分布式调度。通过将 **WorkerAgent** 类加上 `@ray.remote` 装饰器，直接将各设备实例化为 Ray 集群中独立的 Actor 进程。Master 节点只需通过远程方法调用（RPC）即可下发指令，并通过返回的对象引用（Object Ref）统一步节点状态。

独立思考与深度反思 (Highlight)

多 Agent 系统的设计精髓在于“去中心化的执行”与“中心化的监管”相分离。引入 Ray 框架不仅完美解决了本地进程隔离与异步通信的痛点，更为项目未来向真实的跨机器分布式集群平滑迁移奠定了基础。这种“单机模拟验证，无缝拓展至集群”的工程设计思维，是构建高可用分布式架构的最佳实践原则。

6.2 挑战 2：异构硬件特性的拟真与精细化资源博弈

实践困难：最初编写逻辑时，任务对资源的影响仅仅被粗暴地量化为单一的“CPU 占用”和“定值温度飙升”。这导致模拟结果过于刻板，不仅脱离了物理现实，也完全无法体现出现代 AI 大模型推理中 CPU、GPU、NPU 各自截然不同的功耗与发热特性，使得资源博弈失去了深度。

解决办法：针对异构硬件进行多维度的状态拆解与随机化重构。我将节点状态拆分为独立的 CPU、GPU 和 NPU 占用率，并为大模型流水线上的不同环节设计了专属的资源指纹：GPU 负责的“LLM

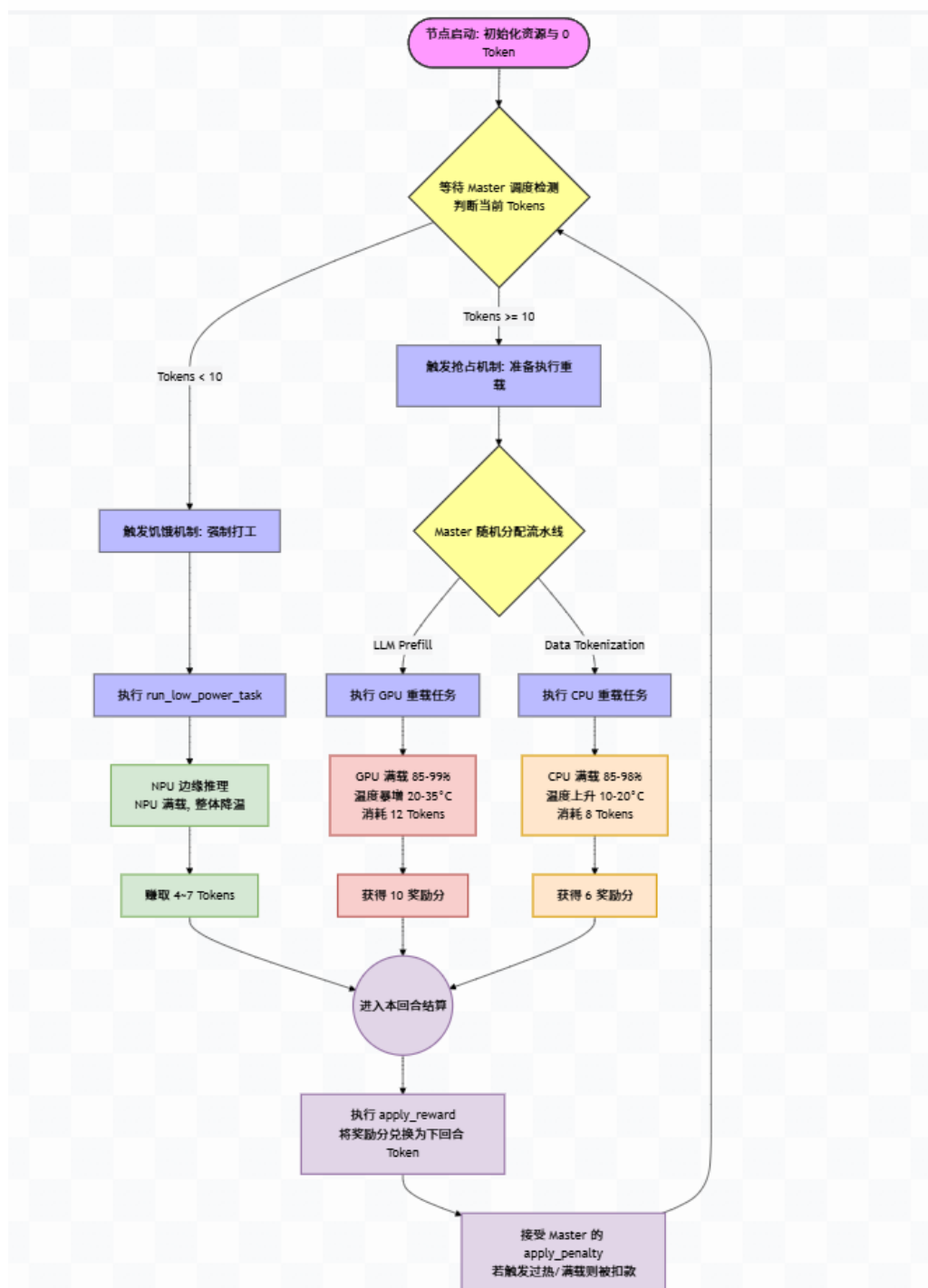


图 1: Worker agent 流程图

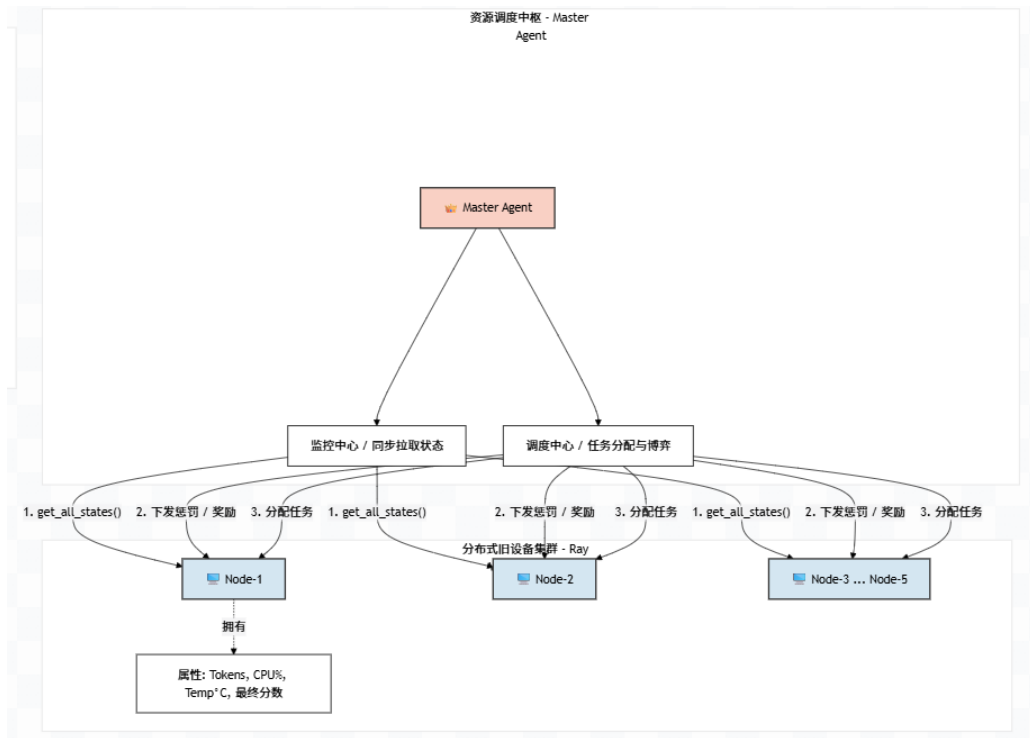


图 2: Master agent 流程图

Prefill”不仅极度消耗 Token（12），还会导致温度剧烈飙升（增加 20 ~ 35°C）；CPU 负责的“Data Tokenization”耗能与发热中等；而 NPU 则能在极低功耗下接管边缘推理赚取 Token，甚至实现物理降温。所有数值均使用 `random.uniform` 引入高斯噪声区间，以模拟真实物理环境的波动。

独立思考与深度反思 (Highlight)

真实物理硬件的瓶颈往往是非线性的黑盒。将死板的单一变量改为多维度的异构“噪声区间”，彻底激活了博弈生态的复杂性。更重要的是，基于硬件异构特性，我将 Master 的“一刀切罚款”升级为了“阶梯式惩罚”（整体过热扣 15 分，GPU 撞墙扣 10 分，CPU 满载扣 5 分）。这种底层的“物理不可靠性”与精细化调控，使得 Master Agent 的调度策略面临更真实的博弈考验，完美模拟了现代系统应对硬件“过热降频（Thermal Throttling）”时的断臂求生机制。

6.3 挑战 3：回合制状态同步与异步执行的时序冲突

实践困难：随着模拟环境加入了随机耗时，不同进程的 Worker 节点任务执行步调产生差异。由于代码是异步调用的，导致 Master 在回合末尾拉取状态时发生了严重的“时序错位”——可能取到了上一回合尚未更新的旧数据。结算日志呈现出状态混乱，导致惩罚与奖励的逻辑判断发生误判，破坏了调度的公平性。

解决办法：在 Master 的调度回合（Round）末尾，强制引入了时间与状态的“同步屏障”（Barrier）机制。在下发完所有异步任务后，首先使用 `time.sleep()` 让主进程短暂挂起等待执行完毕；最核心的改动是，在获取所有节点状态时，强制使用 `ray.get()` 的阻塞特性，确保所有 Actor 的远程状态均已正确更新并返回后，再打印本回合结算面板，随后进入下一轮的博弈。

独立思考与深度反思 (Highlight)

这是分布式系统中经典的“一致性与并发性”冲突。在追求 Agent 并发执行效率的同时，必须在关键的业务节点（即本系统的每一回合结算点）建立强约束的“状态快照”。采用 `ray.get()` 阻塞拉取，虽然牺牲了极少量的主进程等待时间，却换来了整个博弈系统在逻辑调度上的绝对一致性与正确性。若无状态的一致性保证，基于此产生的所有 Token 扣除和奖励都将失去合法依据。在特定环节“以局部同步保全局一致”，是复杂调度系统不可或缺的设计妥协。

7 不足与观察思考

独立思考与深度反思 (Highlight)

物理约束的巧妙投射：实验证明，在这个多 Agent 系统中，“电费 Token”本质上起到了系统“热量缓冲池（Thermal Headroom）”的作用。传统的纯算力调度容易把边缘机器烧毁，而我们利用 Token 作为惩罚介质，一旦 GPU 触发温度墙，立刻剥夺其高负载权限，强制将其切换到 NPU 避险。这完美复刻了现代硬件底层的过热降频（Thermal Throttling）机制，是一种高度安全且健壮的调度范式。

现存不足与未来改进方向：目前的阶梯式惩罚（扣 15/10/5 分）仍偏向于硬编码。真实操作系统的资源调度器（如 Linux 的 CPUFreq）使用的是平滑的 PID 控制或动态电压频率调整（DVFS）。未来可以引入大模型进行更微观的预测性调度，不仅监控温度，还应将大模型推理过程中极为重要的 **KV Cache（显存占用）** 纳入状态向量，使节点资源的博弈维度更加完善。

参考文献

- [1] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., ... & Stoica, I. (2023). *Efficient memory management for large language model serving with pagedattention*. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP 2023)*, pp. 611-626.
- [2] Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., ... & Ji, H. (2024). *A survey on large language model based autonomous agents*. *Frontiers of Computer Science*, 18(6), 1-26.
- [3] Lin, J., Tang, J., Tang, H., Yang, S., Dang, X., & Han, S. (2023). *AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration*. In *Proceedings of Machine Learning and Systems (MLSys 2024)*.
- [4] Zheng, L., Li, Z., Zhang, H., Zhuang, S., Gonzalez, J. E., & Stoica, I. (2022). *Alpa: Automating inter-and intra-operator parallelism for distributed deep learning*. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2022)*, pp. 559-578.
- [5] Chase, H. (2023). *LangChain: Building applications with LLMs through composability*. GitHub repository. <https://github.com/langchain-ai/langchain>

A 附录:github 链接

- 项目代码仓库: <https://github.com/jiling-shenyi/Shujutongxing>

B 附录: 实验结果

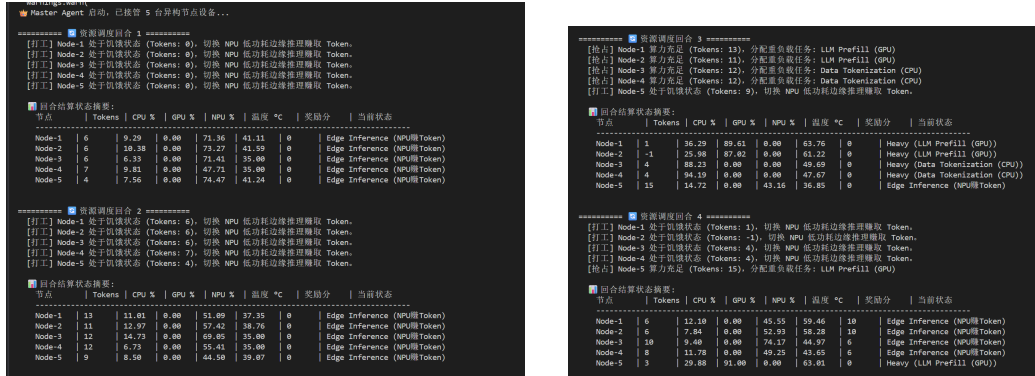


图 3: 1-4 回合

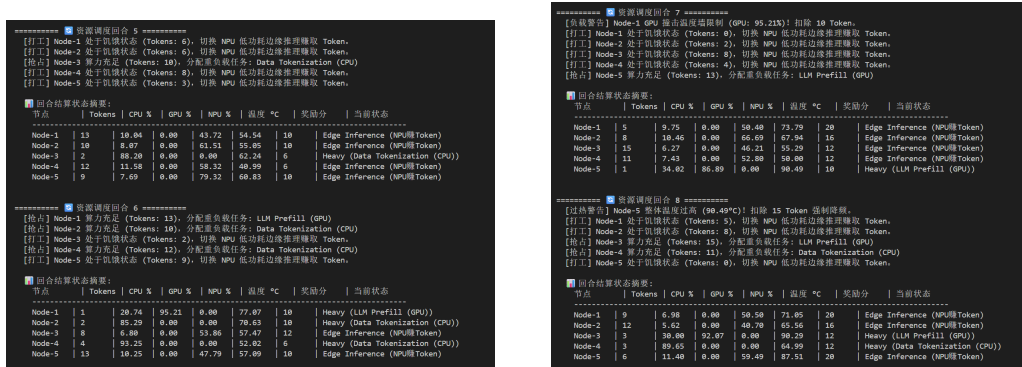


图 4: 5-8 回合

```
===== 资源调度回合 9 =====
[过热警告] Node-3 整体温度过高 (90.25°C)! 扣除 15 Token 强制降频。
[过热警告] Node-5 整体温度过高 (87.51°C)! 扣除 15 Token 强制降频。
[打工] Node-1 处于饥饿状态 (Tokens: 9), 切换 NPU 低功耗边缘推理赚取 Token。
[抢占] Node-2 算力充足 (Tokens: 12), 分配重负载任务: Data Tokenization (CPU)
[打工] Node-3 处于饥饿状态 (Tokens: 0), 切换 NPU 低功耗边缘推理赚取 Token。
[打工] Node-4 处于饥饿状态 (Tokens: 3), 切换 NPU 低功耗边缘推理赚取 Token。
[打工] Node-5 处于饥饿状态 (Tokens: 0), 切换 NPU 低功耗边缘推理赚取 Token。

回合结算状态摘要:
节点 | Tokens | CPU % | GPU % | NPU % | 温度 °C | 奖励分 | 当前状态
-----|-----|-----|-----|-----|-----|-----|-----
Node-1 | 14 | 13.28 | 0.00 | 68.71 | 68.56 | 20 | Edge Inference (NPU赚取Token)
Node-2 | 4 | 90.51 | 0.00 | 0.00 | 79.84 | 16 | Heavy (Data Tokenization (CPU))
Node-3 | 6 | 14.72 | 0.00 | 59.17 | 87.97 | 22 | Edge Inference (NPU赚取Token)
Node-4 | 9 | 8.44 | 0.00 | 78.95 | 60.24 | 18 | Edge Inference (NPU赚取Token)
Node-5 | 7 | 12.63 | 0.00 | 71.49 | 83.76 | 20 | Edge Inference (NPU赚取Token)

===== 资源调度回合 10 =====
[过热警告] Node-3 整体温度过高 (87.97°C)! 扣除 15 Token 强制降频。
[抢占] Node-1 算力充足 (Tokens: 14), 分配重负载任务: Data Tokenization (CPU)
[打工] Node-2 处于饥饿状态 (Tokens: 4), 切换 NPU 低功耗边缘推理赚取 Token。
[打工] Node-3 处于饥饿状态 (Tokens: 0), 切换 NPU 低功耗边缘推理赚取 Token。
[打工] Node-4 处于饥饿状态 (Tokens: 9), 切换 NPU 低功耗边缘推理赚取 Token。
[打工] Node-5 处于饥饿状态 (Tokens: 7), 切换 NPU 低功耗边缘推理赚取 Token。

回合结算状态摘要:
节点 | Tokens | CPU % | GPU % | NPU % | 温度 °C | 奖励分 | 当前状态
-----|-----|-----|-----|-----|-----|-----|-----
Node-1 | 6 | 95.38 | 0.00 | 0.00 | 83.09 | 20 | Heavy (Data Tokenization (CPU))
Node-2 | 9 | 7.67 | 0.00 | 48.20 | 76.08 | 22 | Edge Inference (NPU赚取Token)
Node-3 | 6 | 9.89 | 0.00 | 56.83 | 83.62 | 22 | Edge Inference (NPU赚取Token)
Node-4 | 15 | 5.72 | 0.00 | 57.24 | 55.26 | 18 | Edge Inference (NPU赚取Token)
Node-5 | 13 | 6.32 | 0.00 | 47.80 | 78.93 | 20 | Edge Inference (NPU赚取Token)
```

(a) 9-10

图 5: 9-10 回合